FastSpeech 2 for English and Telugu

Technical report - IIIT Hyderabad RA assignment

7 November 2026

This report covers two FastSpeech 2 text-to-speech models that I trained from scratch: one for English (Svarah) and one for Telugu (IndicTTS). The sections follow the order of the assignment brief. All numbers come from the logs and files in the repository. Listening comments are my own, from a single informal listener.

	English	**Telugu**
Dataset	Svarah: Indian-accented English, 113 speakers	IndicTTS Telugu: one male speaker
Training / validation data	5.00 h (3,417 clips) / 1.00 h (757 clips)	5.00 h (2,625 clips) / 1.00 h (525 clips)
Model	FastSpeech 2, 35.4M parameters, multi-speaker	Same architecture, single speaker
Vocoder	Pretrained HiFi-GAN, fine-tuned on this model's output	Pretrained HiFi-GAN, unchanged
Submitted checkpoint	Step 20,000	Step 20,000

1. Implementation approach and architecture

Starting point. I used the open-source FastSpeech 2 implementation by ming024 (github.com/ming024/FastSpeech2, commit `d4e79eb`, MIT licence) together with the HiFi-GAN vocoder that comes with it. I did not rewrite the network. The model, loss function and learning-rate schedule are unchanged from that code.

How the model works. Text is first converted into phones (speech sounds). An encoder turns the phones into hidden vectors. A *variance adaptor* then predicts three things for every phone: how long it lasts (duration), its pitch (F0) and its loudness (energy). The phone vectors are stretched to the predicted length, and a decoder turns them into a mel-spectrogram. Finally the HiFi-GAN vocoder turns the mel-spectrogram into audio.

- During **training**, the decoder is given the real durations, pitch and energy measured from the recording, and the three predictors learn to guess them.
- During **inference**, only the text (and a speaker for English) is given, so the model must use its own predictions. This is why audio produced during training sounds better than real text-to-speech output.

Changes I made to the reference code:

- **Compatibility fixes** for current library versions (librosa 0.10, CPU-only PyTorch, safe checkpoint loading), plus one upstream bug fix: the energy embedding was being scaled by the pitch control. All changes are in `patches/ming024_fastspeech2_compat.patch`.
- **Silence handling.** The aligner I used (MFA 3) marks pauses as empty intervals, which the original code silently dropped. This would have shifted every duration after the first pause without any error. I relabel these intervals as pauses (`sp`) before feature extraction.
- **Telugu phones.** The original code silently drops any phone it does not know. I added the 49 Telugu phones to its symbol table, and training refuses to start if any phone is unknown.
- **Training loop.** It can resume after a disconnection (important on Colab), saves checkpoints safely, logs losses to CSV and supports mixed precision. It also stores the normalisation statistics and speaker list next to each checkpoint, so inference always uses the same settings as training.
- **Inference front end.** A text-only pipeline converts raw text to phones in the same way as for the training data (Section 4).

Audio settings. 22,050 Hz audio, 1024-point FFT, hop of 256 samples (11.6 ms per frame) and 80 mel bands from 0 to 8 kHz. These match the HiFi-GAN vocoder exactly, and a test checks this.

2. Data, preprocessing and training setup

Splits. For both languages I measured the audio length directly instead of trusting the dataset cards. The splits are fixed (random seed 1234) and grouped by sentence text, so no sentence appears in both training and validation. Tests confirm that training and validation share no clips, audio or sentences. Pitch and energy statistics are computed from the training split only.

English: Svarah (ai4bharat, CC BY 4.0). Svarah contains 9.6 h of Indian-accented English recorded for speech recognition, both read and spontaneous. It has no speaker column, so I used each session's demographic profile as the speaker ID (115 profiles). I removed clips shorter than 1 s, longer than 15 s, too quiet, or with an unusual speaking rate. The final split is 5.00 h for training and 1.00 h for validation, over 113 speakers. The median speaker has only about 1.9 minutes of training audio, so a single-speaker model was not possible and the English model is multi-speaker.

Telugu: IndicTTS Telugu (IIT Madras). The download held 36.7 h from two speakers, not the ~8.7 h on the card. I chose the male speaker: he has 8.1 h of usable clips (15 s or shorter), against 3.6 h for the female speaker, and all his recordings come from one consistent setup. The split is 5.00 h for training and 1.00 h for validation. I rejected 11 transcripts in which a Latin "o" had been typed in place of the Telugu anusvara, instead of correcting them silently.

English preprocessing.

1. Resample the audio from 16 kHz to 22.05 kHz.
2. Write numbers, times and abbreviations as words (for example "10:45 AM" becomes "ten forty five a m"). The same code is used at inference.
3. Look up pronunciations in the LibriSpeech lexicon. A g2p model handles 1,395 unknown words, mostly Indian names and places.
4. Align audio and text with the Montreal Forced Aligner (MFA 3.4.3, pretrained English model). This gives the start and end time of each phone.
5. Extract the training targets with the reference code: mel-spectrogram, phone durations, pitch (WORLD, interpolated through unvoiced parts) and energy, each averaged per phone.
6. Drop 437 clips with an unrealistically long phone, usually an untranscribed "uhh", and replace them from a reserve pool.

Telugu preprocessing. There is no ready-made pronunciation dictionary or alignment model for Telugu, so I built both:

- A **rule-based Telugu-to-phone converter** (`fs2ra/text_te.py`) handles the inherent vowel and virama, vowel signs, anusvara before consonants, vocalic r, visarga and double consonants. It also writes numbers, ordinals and times as Telugu words.
- An **MFA alignment model trained on this Telugu corpus itself**. It ran on Colab (about 100 minutes) and aligned 3,778 of 3,782 clips.

As a sanity check, the speaker's average pitch came out at 190 Hz, which is high for a male voice. Two different pitch trackers agreed (median 179 Hz, no octave errors), so this is his real speaking style, not a bug.

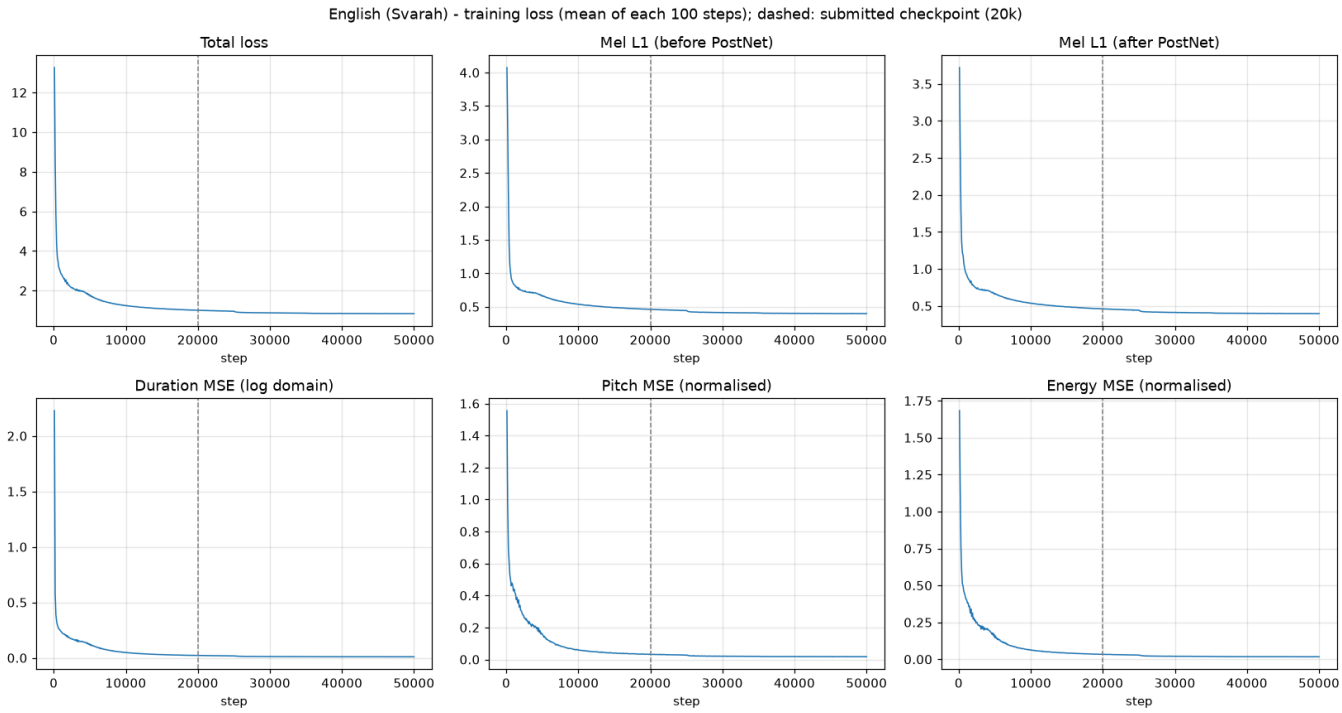
Training configuration (the same for both languages):

Setting	Value
Optimiser	Adam with 4,000 warm-up steps, learning rate x0.3 at 25k, 35k and 45k steps
Batch size	16

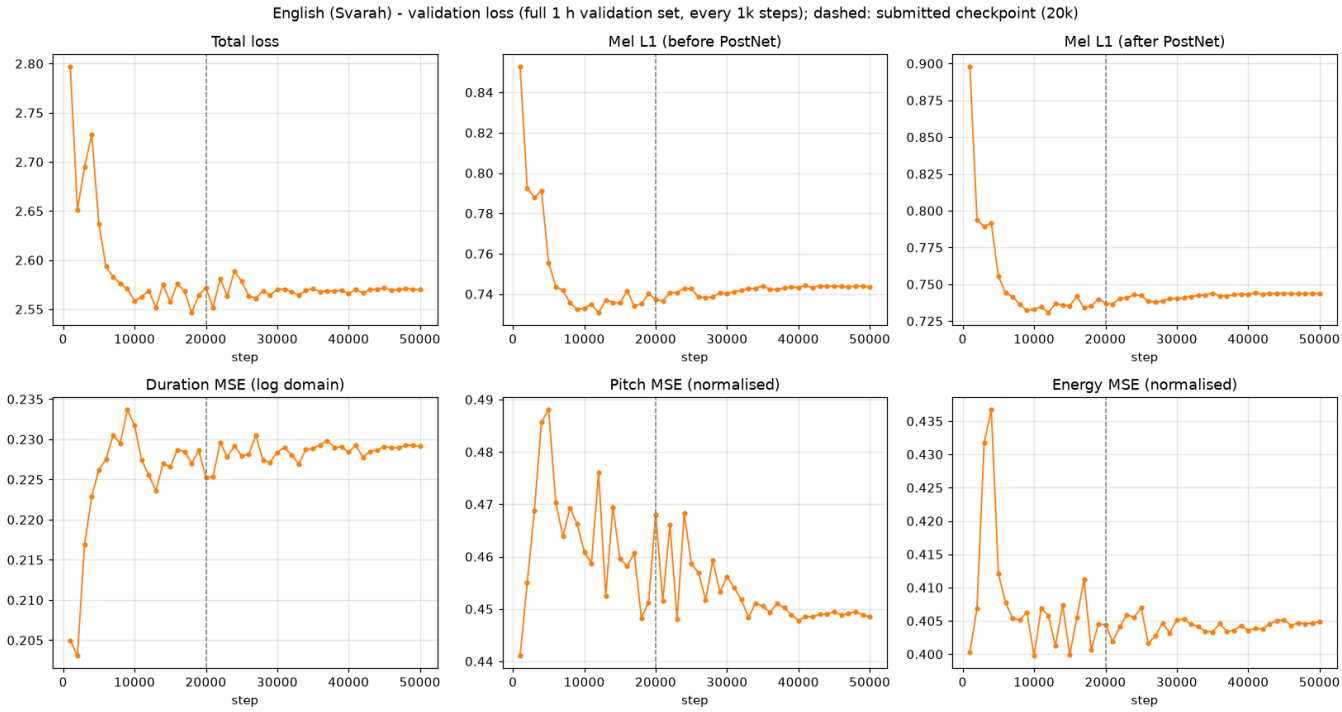
Setting	Value
Training length	50,000 steps
Validation	full 1-hour validation set every 1,000 steps
Hardware	Google Colab GPU: English about 5 hours (T4, then L4), Telugu about 2.5 hours (L4)

3. Training and validation loss curves

English

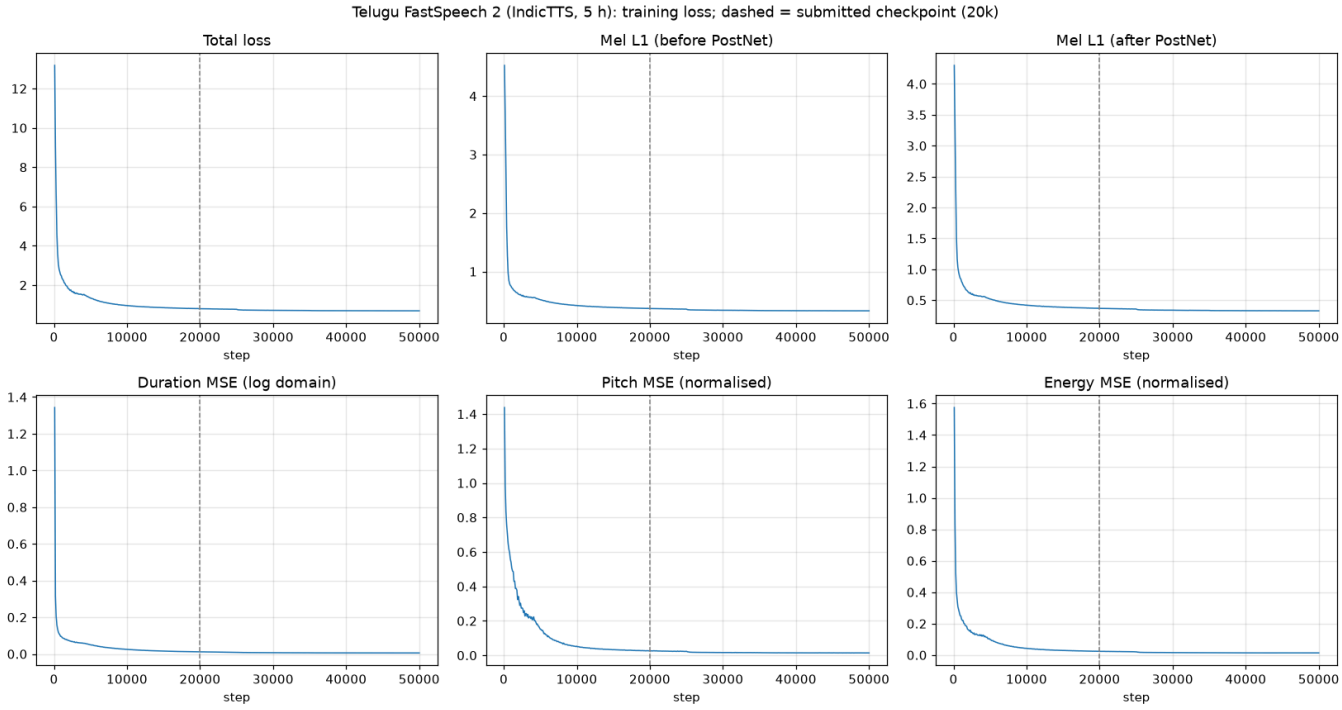


English training loss. The dashed line marks the submitted checkpoint (20k).

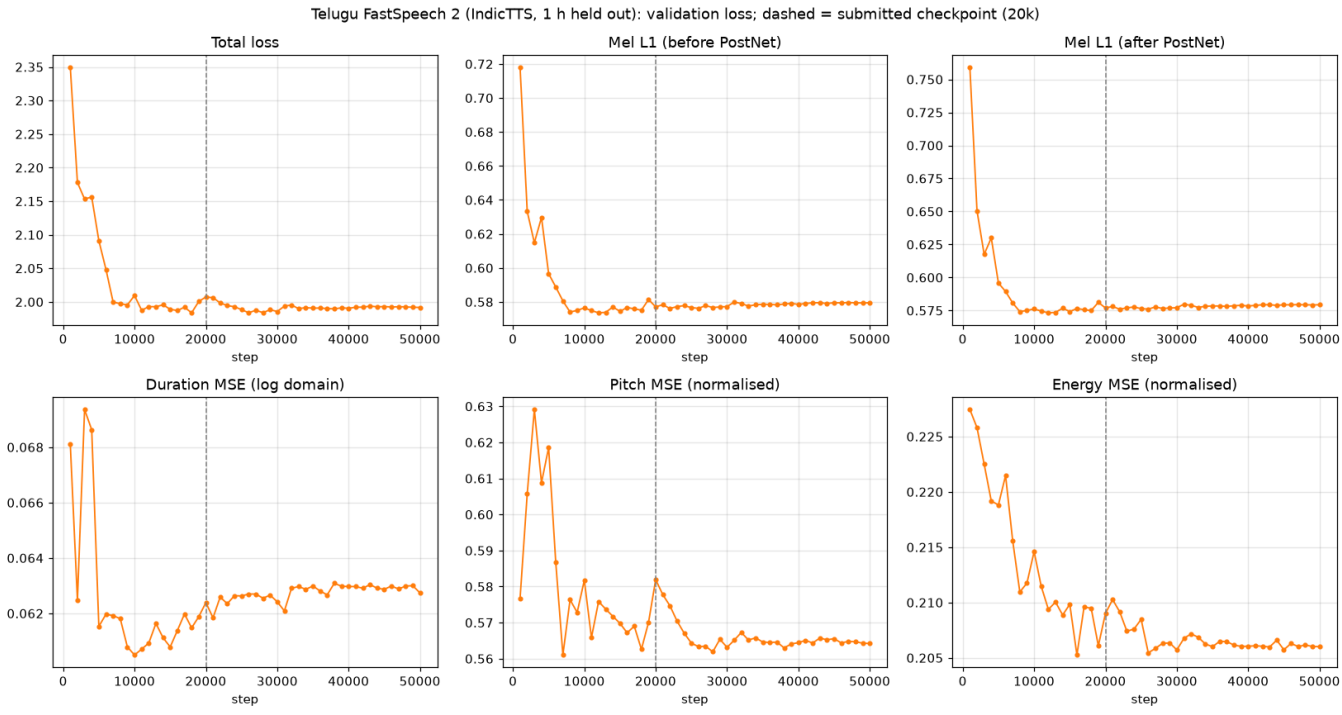


English validation loss, measured every 1,000 steps.

Telugu



Telugu training loss.



Telugu validation loss.

Convergence. Training was stable for both languages, with no NaN values and no loss spikes after warm-up. English training loss fell from 13.3 to 1.01 at 20k steps and 0.84 at 50k. Telugu fell from 13.2 to 0.80 at 20k and 0.69 at 50k.

Overfitting. For both languages the validation loss stops improving after about 8,000 to 10,000 steps (lowest at 18k: English 2.55, Telugu 1.98) and then stays flat, while the training loss keeps falling. This is clear overfitting: the model memorises the training set but does not get better on new sentences. It does not get worse either. With only 5 hours of data, 50,000 steps is more than needed.

Checkpoint choice. The 20k, 30k and 50k checkpoints have almost the same validation loss. I listened to all of them, preferred 20k for both languages, and submitted 20k.

Did the predictors actually learn? I compared each predictor's validation error with a simple baseline (for example, always guessing the speaker's average pitch):

Predictor	English: model vs baseline	Telugu: model vs baseline
Pitch	0.45 vs 0.48	0.58 vs 1.06
Energy	0.40 vs 1.03	0.21 vs 1.09
Duration	0.23 vs 0.26	0.062 vs 0.131

The Telugu model roughly halves the baseline error for all three. The English model learns energy well, but its pitch is barely better than guessing the speaker's average. I explain why in Section 6.

4. Generated speech and pronunciation

All 10 test sentences (5 per language) are in `Results/<Language>/test_generated_audios/`. The exact input text is kept, including the unusual spellings and double spaces. For each word, `inference_summary.json` records the phones that were used.

Front-end decisions:

- **English words with several dictionary pronunciations** use the one that the training speakers actually used most often. For example, "twenty" is pronounced with its T and "parliamentary" with its T, because that is what the aligner found in the training data. My first version picked the dictionary's first entry and produced "twenny" and "parlimenary".
- ***conduct* (E2a noun, E2b verb)** and **పట్టు (T2a, T2b)**. For English, a part-of-speech tagger chooses the pronunciation: CON-duct for the noun, con-DUCT for the verb. The two Telugu words are spelled and pronounced the same, so they get identical phones and only the sentence context differs.
- **Double spaces** (E3 "rhyth m", T3 "అడ విలో") are not spell-corrected, as the brief asks to preserve the text. The extra space is read as a short pause. "rhyth" is not an English word, so the g2p model guesses its pronunciation, and the lone "m" is read as the letter name.
- **Numbers and dates** (E4, T4) are written out as words. Digits never appear in the Telugu training text, so this is the only way the model can say them.
- **Telugu comma (T4).** In the long T4 sentence, the model filled the comma pause with a voiced "aaa" sound at every checkpoint, although every pause in the training data is silent. As a fix, each part of the sentence between punctuation marks is now generated separately, and the parts are joined with silence of the length the model predicted. The pause is now fully silent.

English: speech quality. The English voice is understandable, but it does not sound fully natural.

- **Background buzz.** The first version had a robotic buzz. A listening test showed where it comes from: the original recordings were clean, and real spectrograms passed through the vocoder were also clean. Only the model's own spectrograms buzzed. So the cause is the model's blurred (over-smoothed) spectrogram output, a known weakness of FastSpeech 2, not noise in the data. To reduce it, I fine-tuned the HiFi-GAN vocoder for 3,000 steps on this model's own spectrograms from the English training set. On listening, the buzz was reduced. The old audio is kept in `Results/English/before_vocoder_finetuning/`.
- **Voice choice.** Because the English model has many speakers, I compared how well it reproduces each speaker's own held-out recordings (`scripts/speaker_search.py`). The speaker with the most data (`en_edfd2580`) ranked only 23rd of 89. I listened to five of the best-ranked speakers and judged `en_72a36e45` (ranked 2nd) the best. It also speaks more slowly than the earlier voice (11 to 13.5 vs 12 to 15.5 phones per second). The earlier voice is kept in `Results/English/previous_speaker_en_edfd2580/`.

- **Pronunciation.** The phones given to the model are correct, but the delivery still sounds hesitant, as if the speaker were struggling with some words. In my listening of the first version, the date in E4 sounded rushed ("twel… twenty six") and the noun and verb *conduct* did not sound clearly different. These come from the predicted durations and pitch (Section 5), not from the input phones.

Telugu: speech quality. I judged the 20k Telugu output good for submission. I am not a fluent Telugu speaker, so I could only judge pronunciation roughly. The measurements in Section 5 show a much clearer sentence melody than English.

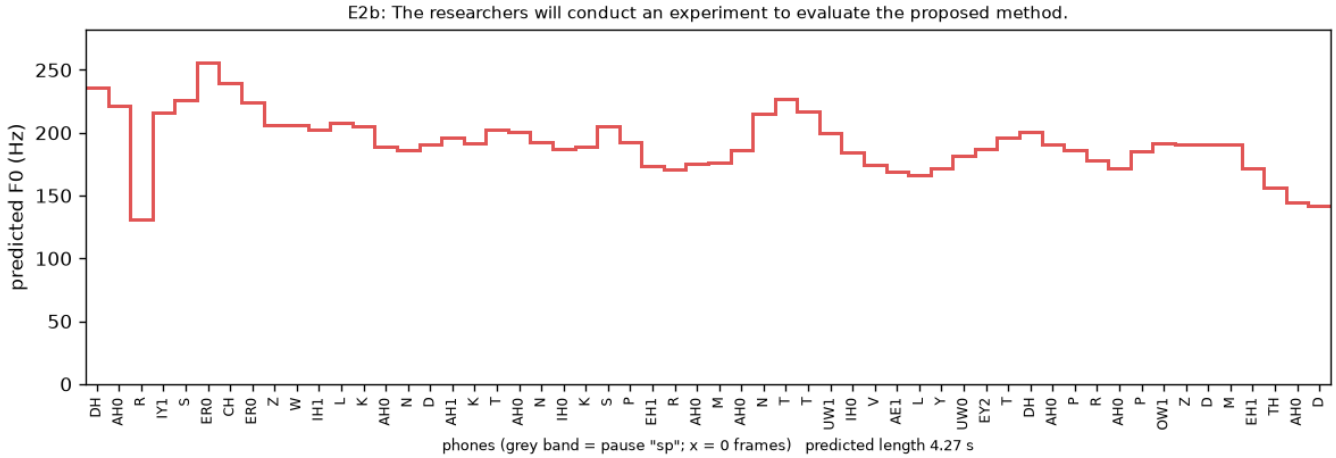
- The model speaks fast. I measured this, and it matches the real speaker's speed (about 15 phones per second). I also generated 10% and 20% slower versions, but preferred the normal speed.
- **T1:** ఔషధాలు sounds like "ushadhalu" instead of "aushadhalu". The phones are correct, but a word-starting ఔ appears only once in the 5 hours of training data, so the model has not learned it. It predicts this vowel much too quietly.
- **T4:** the comma pause is now silent (see above).

5. Variance adaptor plots (duration, pitch, energy)

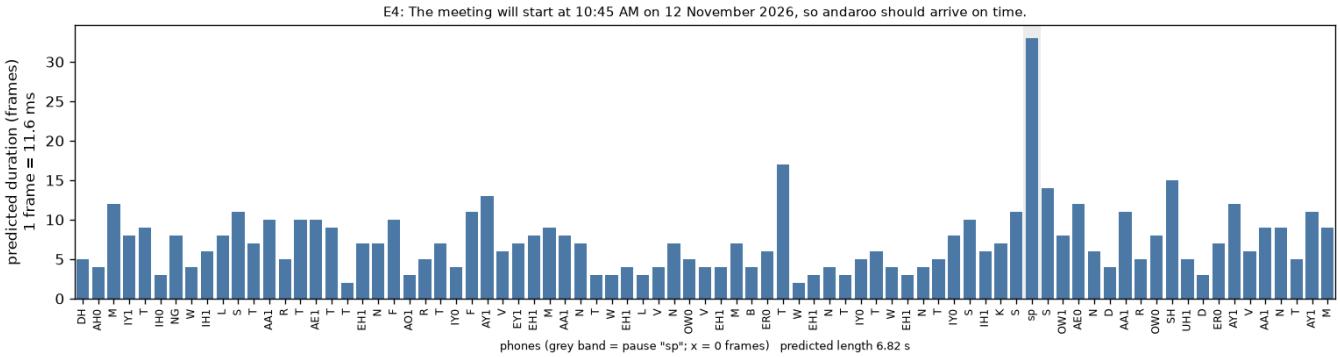
The plots for every test sentence are in `Results/<Language>/variance_adaptor_plots/{duration,pitch,energy}/`. They show the model's predictions, one value per phone. Grey bands mark pauses. One frame is 11.6 ms.

English (speaker `en_72a36e45`):

Sentence	Length	Speed (phones/s)	Pitch: average, spread (range)	What stands out
E1	6.5 s	13.5	201 Hz, ±22 (163-290)	The long word "antidisestablishmentarianism" lasts 2.0 s. The pitch peak lands on an unstressed syllable, not the stressed one.
E2a	4.4 s	11.8	185 Hz, ±21 (102-228)	*conduct* (noun): the stressed vowel is twice as long as the unstressed one (12 vs 6 frames), but not higher in pitch.
E2b	4.3 s	13.1	192 Hz, ±23 (131-256)	*conduct* (verb): the stressed vowel is louder (+60%) but slightly shorter than the unstressed one.
E3	3.3 s	11.3	196 Hz, ±23 (169-254)	A 0.26 s pause at the double space between "rhyth" and "m".
E4	6.8 s	11.7	202 Hz, ±22 (161-261)	0.38 s pause at the comma. The date is squeezed: the stressed vowels in "twelve" and "november" get only 4 frames (46 ms).



E2b predicted pitch. The stressed syllable of *conduct* is only about 7 Hz above the unstressed one.



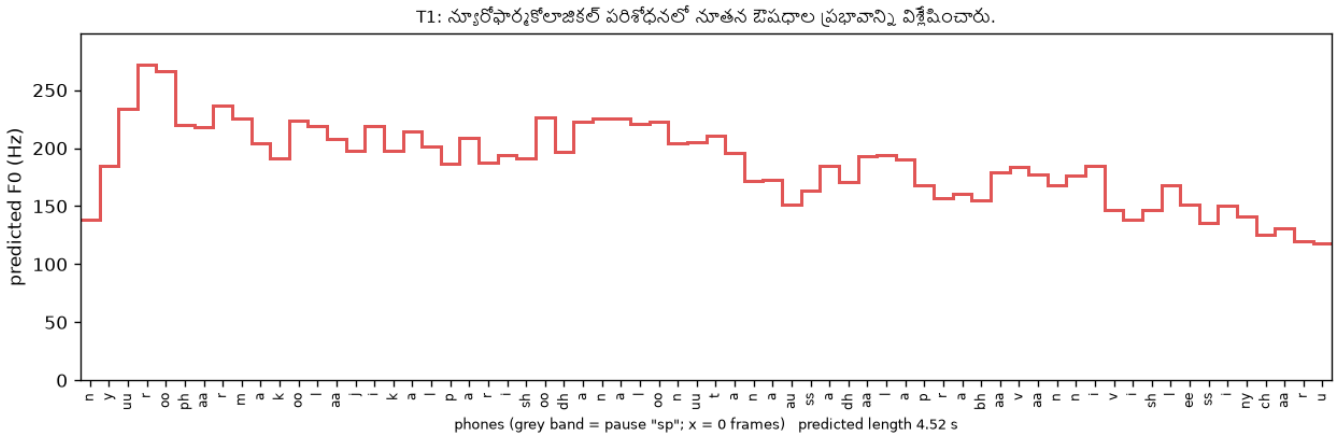
E4 predicted durations. The date words at the end are compressed.

- **Pitch** moves within about ±22 Hz of the speaker's average, but there is no clear sentence melody: it does not fall towards the end of a statement. This matches the weak pitch learning in Section 3 and is a large part of why the voice sounds flat.
- **Stress is weak.** The model marks stressed syllables with only one cue at a time (length in E2a, loudness in E2b), and only slightly. This explains why the noun and verb *conduct* do not sound clearly different.
- **Durations** are reasonable overall: no phone is skipped, and the pauses at the comma and double space are clearly visible. Dates and numbers are compressed, because spoken dates are rare in Svarah.
- **Energy** follows the phones as expected: high on vowels, low on stop closures and pauses. It is the best-learned of the three predictors.

Telugu:

Sentence	Length	Speed (phones/s)	Pitch: average, spread (range)	Pitch at start vs end	What stands out
T1	4.5 s	14.6	188 Hz, ±34 (118-272)	212 to 154 Hz	The రొ vowel is predicted with low energy (Section 4).
T2a	2.1 s	15.0	187 Hz, ±30 (115-234)	201 to 157 Hz	పట్టు: long double-consonant closure (5 + 3 frames).
T2b	2.3 s	16.3	192 Hz, ±35 (120-253)	215 to 160 Hz	పట్టు: closure of 7 + 3 frames. Same phones as T2a, small context difference.

Sentence	Length	Speed (phones/s)	Pitch: average, spread (range)	Pitch at start vs end	What stands out
T3	2.1 s	15.3	182 Hz, ±36 (113-264)	196 to 160 Hz	The double-space pause is only 81 ms, barely audible.
T4	7.4 s	16.6	191 Hz, ±31 (102-260)	199 to 174 Hz	0.23 s comma pause, generated as silence.



T1 predicted pitch. The pitch falls steadily from about 270 Hz to 120 Hz across the sentence.

- **Pitch** has a real sentence melody: every sentence starts high and falls by 25 to 58 Hz towards the end, as natural statements do. The pitch spread (±30 to 36 Hz) is wider than English, which shows the model learned Telugu intonation.
- **Durations** are plausible. The double consonant ట్ట in పట్టు gets a long closure in both T2a and T2b, as it should.
- **Energy** follows vowels and consonants as expected, like English.

6. Comparing English and Telugu: limitations and possible improvements

The same code, model size and training length gave clearly different results. Telugu learned pitch and duration well, while English learned them only weakly. The main reason is the data:

	English (Svarah)	Telugu (IndicTTS)
Purpose of the dataset	Speech recognition	Text-to-speech
Speakers	113 (about 2 min each)	1 (5 hours)
Speaking style	Mostly spontaneous	Read, prepared text
Recording conditions	Varied, 16 kHz	One studio setup

With about 2 minutes per speaker of spontaneous, varied recordings, the English model learns an “average” voice. This gives blurred spectrograms (the buzz) and flat intonation. Telugu has one consistent speaker reading clear sentences, so its prosody is much easier to learn.

Limitations and failures:

- **English blurriness and flat intonation.** Vocoder fine-tuning reduced the buzz, but it cannot add detail that is missing from the spectrogram. The delivery still sounds hesitant.
- **Overfitting.** Both models stop improving after about 10k steps. A smaller model, more regularisation or early stopping would suit 5 hours of data better.

- **Rare sounds.** The model mispronounces sounds it has rarely heard, such as the word-initial ఔ in T1, and spoken dates in English.
- **Long Telugu sentences.** The voiced comma pause in T4 was fixed at inference time by generating the phrases separately. The model itself still has this weakness.
- **Unknown and mixed words.** "rhyth" and the Telugu word "andaroo" in English text depend on g2p guesses, and English phones cannot represent Telugu vowels exactly.
- **Speaker labels.** English speaker IDs are demographic profiles, so a few real speakers may be merged.

What I tried that did not help:

- A stronger vocoder (BigVGAN-v2) reproduced real spectrograms more accurately, but it did not remove the buzz. This confirms the problem lies in the model's predicted spectrograms.
- Simple prosody rules applied after prediction, such as a wider pitch range and slightly slower speech, were tuned on validation data. They made no audible difference, so I did not use them (branch `prosody-experiment`).

Possible improvements:

- **Train with an adversarial (GAN) loss or a vocoder trained jointly with the model** (for example JETS). These are the standard fixes for blurred FastSpeech 2 output, but they need retraining and a change to the architecture.
- **Use English data meant for TTS**, or a subset of Svarah with fewer speakers who have longer, cleaner recordings.
- **Improve the alignments**, for example by adapting the English aligner to Indian-accented speech.
- **Cover rare sounds better**, for example Telugu vowels at the start of words and spoken dates in English, by choosing training clips that contain them.
- **Check intelligibility automatically**, for example by running a speech recogniser on the outputs and measuring its error rate.

7. Reproducibility

Repository: https://github.com/shishirshetty111/fastspeech2-svarah-telugu (full instructions in `project/README.md`).

Software. Python 3.11, PyTorch 2.5, Montreal Forced Aligner 3.4.3, librosa 0.10, pyworld 0.3.5 and g2p-en 2.1.0. The full list is in `requirements.txt`. Training ran on Google Colab GPUs; preprocessing and inference run on a normal CPU.

Configuration files. `project/English/config.yaml` and `project/Telugu/config.yaml` hold all preprocessing, model, training and inference settings. Each checkpoint folder also holds a copy of the config, the normalisation statistics and the speaker list used in training.

Checkpoints (stored with Git LFS): `project/English/checkpoints/20000.pth.tar` (plus the fine-tuned vocoder) and `project/Telugu/checkpoints/20000.pth.tar`.

Commands:

```
# setup
pip install -r requirements.txt
python scripts/setup_upstream.py
python -m pytest tests -q

# data: download, check and split (Svarah needs a Hugging Face token in .env)
python scripts/data/download_datasets.py
python scripts/data/build_inventory.py
python scripts/data/make_splits.py

# English
python project/English/preprocess.py --stage export align features
python project/English/train.py --amp            # resumes automatically; 50k steps
python project/English/inference.py --out_dir project/Results/English

# Telugu (alignment runs on Colab: notebooks/align_telugu_colab.ipynb)
python project/Telugu/preprocess.py --stage export
python project/Telugu/preprocess.py --stage features
python project/Telugu/train.py --amp
python project/Telugu/inference.py --out_dir project/Results/Telugu

# your own sentence
python project/English/inference.py --text "Any English sentence." --out_dir my_outputs
```

Licences. The FastSpeech 2 and HiFi-GAN code is MIT-licensed, and its notices are kept. Svarah is CC BY 4.0 and IndicTTS is under the IIT Madras licence. No dataset audio or transcripts are included in the repository.